

AgentFlame: 面向 AI 编程 Agent 系统效应的语义归因剖析

AgentSight 研究原型

2026 年 6 月 14 日

摘要

AI 编程 agent 的可观测性问题不只是“模型调用了哪个 tool”，而是用户语义意图如何导致真实系统副作用：进程、文件、网络、测试、失败重试和 token 消耗。现有 LLM observability 工具擅长展示单次运行的 trace tree、span duration、tool call 和 cost；传统 profiler 和进程工具擅长聚合系统行为。二者分开时，一个用户常问的问题仍然难答：哪个任务语义造成了哪些重复、沉重或越界的系统足迹？本文提出 AgentFlame，一种语义系统效应剖析模型：本地小模型只给 session、prompt 和 LLM call 生成一个词标签；当前 full run 从 agent-native tool logs 继承这些标签，目标模式则让 tool、shell、child process 与 file/network effect 通过确定性 lineage 继承这些标签；最终以 folded stack 形式聚合为 flamegraph。在本机 AgentSight 仓库的 205 个可读 Codex/Claude 真实 session 上，AgentFlame 标注 2,463 个 prompt 和 90,930 个 LLM call，0 个最终标签失败；它把 167,005 个系统观测折叠成 24,295 条语义系统栈。去掉 session/prompt 语义后，nonsemantic baseline 有 90.219% 的观测权重落入混合多个语义区域的 buckets；flat effect baseline 的混合权重为 90.770%。这些结果支持一个机制性结论：语义帧能分开传统 trace 或 process summary 会合并的 task-effect 区域。本文也明确边界：当前 full run 仍基于 agent-native session 历史；R110 只证明 live AgentSight DB 中被 agent root process 覆盖的 in-scope effects 可 100% join；native session/tool export 和用户任务实验尚未完成，因此不能声称用户效用或完整系统 provenance 已被证明。

1 问题

一个 AI 编程 agent 运行结束后，开发者通常不想逐条回放消息。他们想知道：这次运行哪里最重，哪里绕路，哪里反复试错，哪些文件或网络目标被碰过，哪类用户请求导致最多测试、最多读取、最多失败或最多 token。传统工具各自只覆盖一半。LLM trace 工具能展示 agent、LLM call、tool call、prompt、completion、latency 和 cost。

进程或文件审计工具能展示 git、cargo、rg、sed、docker 等系统行为。Span-duration flamegraph 甚至已经出现在 multi-agent workflow debugging 中，用于看 critical path、parallelism、error cascade 和 tool overhead。

但是这些视图仍难回答跨层问题：

为了哪个用户任务，agent 反复运行了哪些命令、读取了哪些路径、产生了哪些网络或文件副作用？

本文的核心判断是：agent observability 的对象不应只是 span，也不应只是进程，而应是 *task-grounded system effect*。我们把目标栈写成：

```
sessionTag;promptTag;llmcall/tool;process*;effect
```

这个模型把两类传统工具分开的信息合并起来。上层的 session、prompt、LLM call 用本地小模型压缩成一个词；下层 tool、process、file/network effect 不由模型猜测，而通过确定性关联继承语义上下文。最后用 folded stack 聚合重复路径，让宽度表示 event count、effect weight 或 token weight，而不是只表示 span duration。

本文的贡献不应表述为“agent flamegraph”。已有系统已经把 agent spans 画成 flamegraph。本文的贡献是一个更具体的系统归因模型：在 agent-native local histories 中把用户语义意图与系统行为代理接起来，并为 live AgentSight traces 定义 exact provenance 目标，使跨 session 的重复副作用可聚合、可检查、可比较。

2 设计

图 1 展示了 AgentFlame 的设计。语义层非常小：每个 session、prompt 和 LLM call 只生成一个 lowercase word，例如 refactor、review、test、research。这些词不是固定 ontology，也不是 verdict；它们只是给折叠栈提供短、稳定、可搜索的任务帧。

系统层不交给 LLM 判断。Agent-native 历史模式从 Codex/Claude JSONL 中恢复 tool 类型、命令入口、状态、路径组和粗 effect 类别。AgentSight exact 模式的目标输入是：

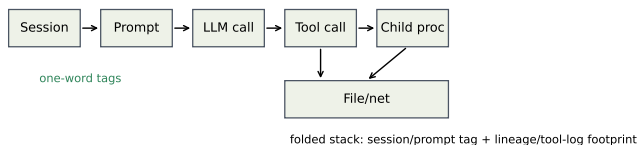


图 1: AgentFlame 的归因模型。小模型只标注 session、prompt、LLM-call 语义；tool/process/file/network effects 通过确定性 lineage 继承语义帧并折叠成 stacks。

```

tool_call -> shell -> child process ->
file/network effect

```

每个 in-scope effect 必须能追溯到 tool call 和 prompt ancestry。如果只能用 PID 或时间戳模糊匹配，就不能作为强 provenance claim。因此，语义由上层继承，系统事实由 instrumentation 或 session parser 产生。

2.1 为什么是一个词

一个词标签有三个工程优点。第一，火焰图帧必须短；长摘要会让图不可读。第二，标签只用于导航和聚合，不承担安全、正确性或因果判决。第三，一个词标签容易本地化和缓存化。当前实现对 session/prompt/LLM 文本各处理一次，后续数十万条系统事件只做普通程序关联和聚合。

这也约束了论文 claim。AgentFlame 不证明标签是真实意图 ontology。它先证明一个更可审计的机制：加入这些短语义帧后，传统 baseline 会合并掉的系统效应是否被分开。

2.2 Stack 语法

系统效应栈形如：

```
project;agent;session;prompt;call:tool/...;process*;effect;path/domain;status
```

Token 栈形如：

```
project;agent;session;prompt;call:llm/...;model;kind;file/network effect
```

完整视图保留 session、prompt、LLM-call 语义。消融视图删除部分语义轴：session-system、prompt-system、llm-token 等。所有视图共享同一事实表；投影不应改变总权重。

3 实现

AgentFlame 现在是独立 Rust CLI，位于仓库 `agentflame/`。它使用 `normalize-chat-sessions` 读取本地 Codex 和 Claude JSONL 历史，调用 llama.cpp-compatible HTTP server 生成标签，输出 `agentflame.json`、`tags.json`、`folded stack` 文件、SVG 和 HTML dashboard。默认输出路径是：

```
.agentsight/agentflame/latest/
```

本次 full run 的命令为：

```

cargo run --manifest-path
agentflame/Cargo.toml -- run --project-root
. --scan-files 10000 --max-sessions
10000 --llama-url http://127.0.0.1:18080
--model local --timeout 60 --out
.agentsight/agentflame/latest

```

模型为本地 `qwen2.5-3b-instruct-q4_k_m.gguf`，通过 llama.cpp server 运行，temperature 为 0，并用 grammar 约束输出一个词。AgentFlame 没有 heuristic fallback：如果 LLM server 缺失，或模型重试后仍不能输出合法一词标签，run 失败。

3.1 隐私与可复现

原始 agent trace 不提交。`agentflame.json` 默认包含 redacted previews、hash、tag、计数和路径组；`tags.json` 保存 tag cache 和 LLM provenance，不保存原始 prompt 文本。本次 run 遇到一个 root-owned Claude JSONL，不可读；系统没有要求 root 权限读取，而是跳过并写入 warnings。这让覆盖范围可审计，而不是静默假装完整。

3.2 与 AgentSight 的关系

AgentFlame 的第一模式是零 instrumentation 的历史分析。AgentSight 的特点是运行时 exact provenance: `tool_call -> shell -> child process ->` `file/network effect`。正确集成后，AgentFlame 负责语义帧，AgentSight 负责系统效应事实，两者通过 tool/session/prompt ancestry 连接。当前 full run 还不是 live exact-effect 结果。R110 增加了一个 live in-scope smoke，但它仍通过 harness 补 session/tool envelope；因此 C4 仍是 partial。

表 1: 全量本机 AgentSight session 运行指标。

指标	数值
Readable sessions	205
Codex / Claude / Claude-subagent	78 / 50 / 77
Raw tool events	130,632
Raw LLM events	90,930
Prompt rows	2,463
Unique prompt tags	303
LLM-call tags	90,930
Unique LLM-call tags	1,250
Tag requests	93,598
Cache hits	64,297
llama.cpp HTTP calls	29,302
Final tag failures	0

4 评估

4.1 RQ1: 本地小模型能否全量标注？

表 1 给出 full run 结果。AgentFlame 分析了 205 个可读 session，其中 Codex 78 个、Claude 50 个、Claude subagent 77 个。它处理 2,463 个 prompt rows 和 90,930 个 LLM-call events。标签器共有 93,598 个请求，64,297 个命中 cache，29,302 次真实 llama.cpp HTTP calls，0 个最终失败。Prompt 和 LLM-call tag 的非法计数均为 0。这说明一词语法和本地 3B 模型路径在真实历史规模上可运行。

这个结果不等于标签语义充分。例如某些 tag 明显带噪声：agentsightsm、testcodex、bashoutput。因此 RQ1 支持的是 feasibility 和 syntax claim；tag adequacy 需要单独实验。

4.2 RQ2: 语义帧是否分割 baseline 混合？

Full run 产生 167,005 个 system observations，并折叠成 24,295 条 unique semantic system stacks，压缩率为 6.874 倍。关键不是压缩本身，而是 baseline mixing。当删除 session/prompt 语义帧后，nonsemantic folded baseline 有 4,209 个 mixed buckets，覆盖 150,670 个观测，也就是 90.219% 的系统权重。如果进一步退化成 flat effect baseline，mixed weight 为 151,590，占 90.770%。

这为回应“传统工具也能看出来”的质疑提供了机制性证据。传统 flat summary 可以告诉我们 sed read 有 25,755 次、rg read 有 15,336 次、cargo test 有 2,903

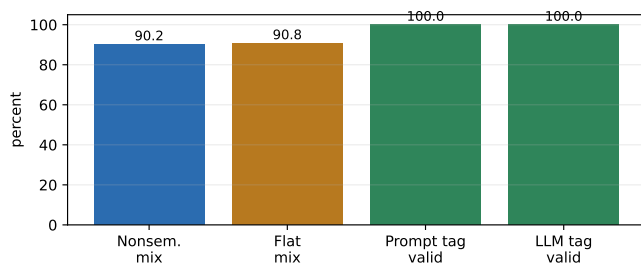


图 2: 当前机制结果。图由 docs/visexp/paper/make_figures.py 从 .agentsight/agentflame/latest/agentflame.json 重新生成；它反映 Rust full-run 输出，而不是旧的 Python sampled-run artifact。

表 2: 语义栈与 baseline mixing。

指标	数值
System observations	167,005
Unique semantic system stacks	24,295
Semantic compression	6.874x
Max stack reuse	6,004
Nonsemantic mixed buckets	4,209
Nonsemantic mixed weight	90.219%
Flat mixed buckets	4,051
Flat mixed weight	90.770%

次。但是它不知道这些行为分别来自 refactor、review、research、design 还是 test prompt 区域。AgentFlame 的价值是把 heavy/repeated system effects 按任务语义拆开。

4.3 Case Study: 能看出什么

本次 full run 中，最大的 semantic stack 是：

```
project:agentsight;agent:codex;session:refactor;prompt:refa
```

权重为 6,004。另一个明确的系统例子是 cargo test。Flat summary 只会显示 2,903 次 successful cargo test。Nonsemantic stack 会保留 call:tool/shell;process:cargo;effect:test;status:ok，但仍把不同语义区域混在一起。Semantic stack 将它拆成多个区域，包括 review/review、refactor/refactor、research/explain、research/refactor、design/review 等。

这类结果生成可供检查的候选分解，用来辅助三类 developer questions：

- 哪个任务导致最多重复测试或读取？
- 哪些 prompt tag 反复触碰同一组路径或域名？
- 哪些失败行为集中在特定 semantic region，而不是全局系统问题？

Trace tree 能回放某次调用，span flamegraph 能看 duration bottleneck，flat summary 能看重命令。但它们都不直接给出跨 session 的 task-effect 聚合。

4.4 RQ3: exact lineage 是否成立？

当前答案是：live in-scope smoke 部分成立，native export 仍不足。docs/visexp/effect_lineage_smoke.py 已经能在 AgentSight-shaped fixture 上把 process/file/network effects 连到 session/tool/prompt ancestry，并输出 exact-effect folded stacks。R110 进一步导出了 3 个真实 AgentSight SQLite DB，使用 docs/visexp/live_lineage_harness.py 给检测到的 Codex/Claude root processes 增加 agent-run session/tool envelope，并用本地 llama.cpp 3B 模型给 root prompt 打一个词标签。在这些被 root process family 覆盖的 in-scope effects 上，checker 验证 182/182 joined，0 orphans，join rate 为 100.0%。

这个结果比 fixture 强，但还不是完整 C4。原因是当前 DB export 本身没有 materialized session/tool rows；harness 只补上 envelope，不伪造底层 process/file/network events。同时 318 个 raw effects 中有 136 个不在检测到的 agent roots 范围内。OSDI 版本必须把 envelope 移入 collector native export，并在更多任务上报告 in-scope coverage、orphan rate、child-process depth、path/domain specificity 和 redaction failures。

4.5 RQ4: 用户效用是否成立？

当前答案是不成立，或者更准确地说：未验证。已有 task packet、answer key 和 scorer 原型，但没有真实参与者响应。下一步需要把 baseline 设为 trace tree、span-duration flamegraph、flat process summary、nonsemantic folded stack 和 semantic folded stack，比较 accuracy、time、false positives 和 confidence。如果这个实验失败，AgentFlame 仍可能是专家 forensic 工具，但不能写成提升普通开发者效率。

4.6 RQ5: 小模型成本和标签质量

Full run 证明 3B 模型可用，但没有证明 0.6B 或 1B 足够。已有 model benchmark 入口可以跑 0.6B/1B/3B latency、success 和 invalid rate。OSDI 版本还需要人工 adequacy labels。一词标签应该被定位为 lossy navigation frames，而不是 semantic truth。

5 相关工作

Flamegraphs 与 profiling. Brendan Gregg 的 flamegraph 让用户通过宽度定位 hot stacks。Grafana/Pyroscope、Datadog、Sentry 等系统已经把 flamegraph 用于 CPU、内存、profile 和 traces。AgentFlame 借用 folded stack 表示，但 stack frame 来自 agent semantic context 与 system-effect lineage，而不是函数调用栈。

Distributed tracing 与 agent observability. OpenTelemetry/Dapper 风格 tracing 把一次请求表示为 span tree。LangSmith、Langfuse、Phoenix、AgentOps 等工具已经记录 agent、LLM、tool、retrieval、cost 和 latency。SigNoz/Inkeep 公开展示了 multi-agent workflow 的 span-duration flamegraph。这些系统擅长解释单次 workflow 如何执行；AgentFlame 的目标是跨 session 聚合“哪个语义任务造成哪些系统副作用”。

Token/context profiling. Context 和 token profiling 工具关注 prompt/context window 中的热点。AgentFlame 包含 token flamegraph，但当前只作为 source-local accounting。论文主贡献是 token/semantic/tool/process/effect 共享同一 folded-stack 语法，而不是 token cost 本身。

6 局限与下一步

当前版本仍不到 OSDI weak accept。第一，exact lineage 是最强系统贡献，但当前只有 live in-scope smoke，还没有 native AgentSight full-run export 证据。第二，用户效用没有实验。第三，标签 adequacy 没有人工评估。第四，当前结果来自一个本机仓库，且历史是 observational，不是 paired benchmark。

最短补强路径是：

1. 运行 5-10 个真实 agent tasks under AgentSight，产出 exact effect join/orphan table。

2. 在同一批 traces 上生成 trace tree、span-duration flamegraph、flat summary、nonsemantic stack、semantic stack 五种视图。
3. 用 12-20 个 forensic questions 运行开发者任务 benchmark。
4. 对 200 个 session/prompt/LLM fragments 跑 0.6B/1B/3B tagger benchmark 和人工 adequacy labels。

- [8] Arize Phoenix. Tracing documentation. <https://arize.com/docs/phoenix>.

7 结论

AgentFlame 的核心不是再画一张 agent flamegraph, 而是把用户语义意图、LLM/tool 历史和系统副作用放入同一个可折叠、可聚合、可审计的栈模型。当前 full run 说明该模型在真实本地 agent 历史上可运行, 并且语义帧能分开 nonsemantic 和 flat baselines 大量混合的系统效应。但顶会版本必须继续证明 live exact lineage 和用户任务收益。在当前证据下, 最诚实的论文定位是: 一个有完整系统化方向和真实 characterization 的语义系统效应 profiler, 而不是已经完成的 OSDI artifact。

参考文献

- [1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- [2] Benjamin H. Sigelman et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report, 2010.
- [3] SigNoz. Understanding Flame Graphs for Visualizing Distributed Tracing. <https://signoz.io/blog/flamegraphs/>.
- [4] SigNoz. How Inkeep Monitors Their AI Agent Framework with SigNoz. <https://signoz.io/blog/inkeep-ai-agent-monitoring/>.
- [5] Datadog. Trace View. https://docs.datadoghq.com/tracing/trace_explorer/trace_view/.
- [6] LangChain. LangSmith tracing documentation. <https://docs.langchain.com/langsmith/>.
- [7] Langfuse. Observability documentation. <https://langfuse.com/docs/observability>.